SQL & Database Architecture Documentation

System: Live Stock & Cryptocurrency Portfolio Management System | **Database:** PostgreSQL 15+ | **Schema Verification:** 100% ER Compliant

This manual documents the comprehensive Data Definition Language (DDL), Data Manipulation Language (DML), Analytical / Reporting queries, and PL/pgSQL Stored Procedures, Functions, and Triggers comprising the system's database architecture.

Table of Contents & Architecture Summary

• **Part 1: Table Creation (DDL) Queries** — Items 1 to 20 (Tables, Constraints, Indexes)
• **Part 2: Insert, Update, and Delete (DML) Queries** — Items 21 to 44 (CRUD Operations)
• **Part 3: SELECT, Analytical, and Reporting Queries** — Items 45 to 61 (Aggregations, PnL, Feeds)
• **Part 4: PL/pgSQL Procedures, Functions & Triggers** — Items 62 to 70 (Automation, State Sync)

Coverage & Metrics

• **Total Database Entities:** 15 Tables, 14 Indexes
• **Total SQL Specifications:** 70 Documented Objects
• **PL/pgSQL Constructs:** 2 Triggers, 7 Stored Functions
• **Status:** All constructs mapped to active schema

PART 1: TABLE CREATION (DDL) QUERIES

1. Enable UUID Extension

```
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";
```

Purpose: Enables the UUID generation functions in the database to generate unique identifiers for primary keys.

ER Diagram Check: ■ Matches

2. Create Currency Rates Table

```
CREATE TABLE currency_rates (
    currency_code VARCHAR(10) PRIMARY KEY,
    rate_to_usd NUMERIC(18, 6) NOT NULL CHECK (rate_to_usd > 0),
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT NULL
);
```

Purpose: Stores reference exchange rates for world currencies relative to USD.

ER Diagram Check: ■ Matches

3. Create Users Table

```
CREATE TABLE users (
    user_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    base_currency VARCHAR(10) DEFAULT 'USD' NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT NULL,
    CONSTRAINT fk_users_currency FOREIGN KEY (
        base_currency) REFERENCES currency_rates (currency_code) ON UPDATE CASCADE
);
```

Purpose: Stores user identities, credentials, base currency preferences, and creation timestamps.

ER Diagram Check: ■ Matches

4. Create Exchanges Table

```
CREATE TABLE exchanges (
    exchange_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    name VARCHAR(100) NOT NULL,
    country VARCHAR(100) NOT NULL,
    url VARCHAR(500)
);
```

Purpose: Stores financial exchanges and trading venues where assets are listed.

ER Diagram Check: ■ Matches

5. Create Assets Table

```
CREATE TABLE assets (
    asset_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    symbol VARCHAR(20) UNIQUE NOT NULL,
    name VARCHAR(150) NOT NULL,
    type VARCHAR(20) NOT NULL CHECK (type IN ('crypto', 'stock')),
    network VARCHAR(100),
    contract_address VARCHAR(255),
    ticker_exchange VARCHAR(50),
    sector VARCHAR(100),
    exchange_id UUID,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT NULL,
    CONSTRAINT fk_assets_exchange FOREIGN KEY (exchange_id) REFERENCES exchanges (
        exchange_id) ON DELETE SET NULL
);
```

Purpose: Catalogs global tradeable assets with polymorphic fields for crypto networks and stock sectors.

ER Diagram Check: ■ Matches

6. Create Ticker Stats Table

```
CREATE TABLE ticker_stats (
    asset_id UUID PRIMARY KEY,
    price_per_unit NUMERIC(18, 6) NOT NULL DEFAULT 0.0,
    volume_24h NUMERIC(24, 2) DEFAULT 0.0,
    high_24h NUMERIC(18, 6) DEFAULT 0.0,
    low_24h NUMERIC(18, 6) DEFAULT 0.0,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT NULL,
    CONSTRAINT fk_ticker_stats_asset FOREIGN KEY (asset_id) REFERENCES assets (
        asset_id) ON DELETE CASCADE
);
```

Purpose: Stores 1:1 24-hour market metrics and live statistics for cataloged assets.

ER Diagram Check: ■ Matches

7. Create Portfolios Table

```
CREATE TABLE portfolios (
    portfolio_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID NOT NULL,
    name VARCHAR(100) NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT NULL,
    CONSTRAINT fk_portfolios_user FOREIGN KEY (user_id) REFERENCES users (
        user_id) ON DELETE CASCADE
);
```

Purpose: Defines user portfolios that aggregate holdings and investment positions.

ER Diagram Check: ■ Matches

8. Create Holdings Table

```
CREATE TABLE holdings (
    holding_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    portfolio_id UUID NOT NULL,
    asset_id UUID,
    symbol VARCHAR(20) NOT NULL,
    name VARCHAR(150) NOT NULL,
    type VARCHAR(20) NOT NULL CHECK (type IN ('crypto', 'stock')),
    quantity NUMERIC(24, 8) NOT NULL CHECK (quantity > 0),
    average_buy_price NUMERIC(18, 6) NOT NULL CHECK (average_buy_price >= 0),
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT NULL,
    CONSTRAINT fk_holdings_portfolio FOREIGN KEY (
        portfolio_id) REFERENCES portfolios (portfolio_id) ON DELETE CASCADE,
    CONSTRAINT fk_holdings_asset FOREIGN KEY (asset_id) REFERENCES assets (
        asset_id) ON DELETE SET NULL,
    CONSTRAINT uq_portfolio_symbol UNIQUE (portfolio_id, symbol)
);
```

Purpose: Stores individual asset positions held within a portfolio with purchase cost bases.

ER Diagram Check: ■ Matches

9. Create Transactions Table

```
CREATE TABLE transactions (
    transaction_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID NOT NULL,
    portfolio_id UUID NOT NULL,
    asset_id UUID,
    symbol VARCHAR(20) NOT NULL,
    type VARCHAR(20) NOT NULL CHECK (type IN ('BUY', 'SELL', 'TRANSFER_IN',
        'TRANSFER_OUT')),
    quantity NUMERIC(24, 8) NOT NULL CHECK (quantity > 0),
    price_at_transaction NUMERIC(18,
        6) NOT NULL CHECK (price_at_transaction >= 0),
    network_hash VARCHAR(255),
    timestamp TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT NULL,
    CONSTRAINT fk_tx_user FOREIGN KEY (user_id) REFERENCES users (user_id) ON
        DELETE CASCADE,
    CONSTRAINT fk_tx_portfolio FOREIGN KEY (portfolio_id) REFERENCES portfolios (
        portfolio_id) ON DELETE CASCADE,
    CONSTRAINT fk_tx_asset FOREIGN KEY (asset_id) REFERENCES assets (asset_id) ON
        DELETE SET NULL
);
```

Purpose: Maintains an immutable ledger of buy, sell, and transfer executions.

ER Diagram Check: ■ Matches

10. Create Portfolio Performance Table

```
CREATE TABLE portfolio_performance (
    performance_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    portfolio_id UUID NOT NULL,
    total_value NUMERIC(18, 2) NOT NULL,
    total_profit_loss NUMERIC(18, 2) NOT NULL,
    total_profit_loss_pct NUMERIC(8, 4) NOT NULL,
    timestamp TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT NULL,
    CONSTRAINT fk_perf_portfolio FOREIGN KEY (
        portfolio_id) REFERENCES portfolios (portfolio_id) ON DELETE CASCADE
);
```

Purpose: Records periodic valuation and profit/loss snapshots for historical tracking.

ER Diagram Check: ■ Matches

11. Create Watchlists Table

```
CREATE TABLE watchlists (
    watchlist_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID NOT NULL,
    name VARCHAR(100) NOT NULL,
    creation_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT NULL,
    CONSTRAINT fk_watchlists_user FOREIGN KEY (user_id) REFERENCES users (
        user_id) ON DELETE CASCADE
);
```

Purpose: Stores custom user watchlists for tracking market symbols.

ER Diagram Check: ■ Matches

12. Create Watchlist Items Table

```
CREATE TABLE watchlist_items (
    item_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    watchlist_id UUID NOT NULL,
    symbol VARCHAR(20) NOT NULL,
    asset_id UUID,
    added_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT NULL,
    CONSTRAINT fk_wl_items_watchlist FOREIGN KEY (
        watchlist_id) REFERENCES watchlists (watchlist_id) ON DELETE CASCADE,
    CONSTRAINT fk_wl_items_asset FOREIGN KEY (asset_id) REFERENCES assets (
        asset_id) ON DELETE SET NULL,
    CONSTRAINT uq_watchlist_symbol UNIQUE (watchlist_id, symbol)
);
```

Purpose: Represents tracked asset entries associated with a watchlist.

ER Diagram Check: ■ Matches

13. Create Alerts Table

```
CREATE TABLE alerts (
    alert_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID NOT NULL,
    symbol VARCHAR(20) NOT NULL,
    asset_id UUID,
    threshold NUMERIC(18, 6) NOT NULL CHECK (threshold > 0),
    direction VARCHAR(10) NOT NULL CHECK (direction IN ('ABOVE', 'BELOW')),
    enabled BOOLEAN DEFAULT TRUE NOT NULL,
    triggered_at TIMESTAMP WITH TIME ZONE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT NULL,
    CONSTRAINT fk_alerts_user FOREIGN KEY (user_id) REFERENCES users (user_id) ON
        DELETE CASCADE,
    CONSTRAINT fk_alerts_asset FOREIGN KEY (asset_id) REFERENCES assets (
        asset_id) ON DELETE SET NULL
);
```

Purpose: Defines price threshold trigger rules created by users.

ER Diagram Check: ■ Matches

14. Create Notifications Table

```
CREATE TABLE notifications (
    notification_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    alert_id UUID NOT NULL,
    user_id UUID NOT NULL,
    channel VARCHAR(20) NOT NULL CHECK (channel IN ('EMAIL', 'SMS', 'PUSH',
        'IN_APP')),
    status VARCHAR(20) NOT NULL CHECK (status IN ('PENDING', 'SENT', 'FAILED')),
    sent_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT NULL,
    CONSTRAINT fk_notif_alert FOREIGN KEY (alert_id) REFERENCES alerts (
        alert_id) ON DELETE CASCADE,
    CONSTRAINT fk_notif_user FOREIGN KEY (user_id) REFERENCES users (user_id) ON
        DELETE CASCADE
);
```

Purpose: Stores notification delivery history for triggered price alerts.

ER Diagram Check: ■ Matches

15. Create Price History Table

```
CREATE TABLE price_history (
    history_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    symbol VARCHAR(20) NOT NULL,
    asset_id UUID,
    price NUMERIC(18, 6) NOT NULL,
    volume NUMERIC(24, 2),
    timestamp TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT NULL,
    CONSTRAINT fk_history_asset FOREIGN KEY (asset_id) REFERENCES assets (
        asset_id) ON DELETE SET NULL
);
```

Purpose: Stores historical timeseries price points for charting and analysis.

ER Diagram Check: ■ Matches

16. Create News Articles Table

```
CREATE TABLE news_articles (
    article_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    headline VARCHAR(500) NOT NULL,
    source VARCHAR(100) NOT NULL,
    url VARCHAR(1000) NOT NULL,
    published_at TIMESTAMP WITH TIME ZONE NOT NULL
);
```

Purpose: Stores market financial news headlines and metadata.

ER Diagram Check: ■ Matches

17. Create News Article Assets Junction Table

```
CREATE TABLE news_article_assets (
    article_id UUID NOT NULL,
    asset_id UUID NOT NULL,
    PRIMARY KEY (article_id, asset_id),
    CONSTRAINT fk_naa_article FOREIGN KEY (article_id) REFERENCES news_articles (
        article_id) ON DELETE CASCADE,
    CONSTRAINT fk_naa_asset FOREIGN KEY (asset_id) REFERENCES assets (
        asset_id) ON DELETE CASCADE
);
```

Purpose: Models the many-to-many relationship linking news articles to mentioned assets.

ER Diagram Check: ■ Matches

18. Create Price Cache Table

```
CREATE TABLE price_cache (
    cache_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    symbol VARCHAR(20) NOT NULL,
    asset_type VARCHAR(20) NOT NULL CHECK (asset_type IN ('crypto', 'stock')),
    price_usd NUMERIC(18, 6) NOT NULL,
    high_24h NUMERIC(18, 6),
    low_24h NUMERIC(18, 6),
    volume_24h NUMERIC(24, 2),
    change_24h NUMERIC(8, 4),
    fetched_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT NULL,
    expire_at TIMESTAMP WITH TIME ZONE NOT NULL,
    metadata JSONB,
    CONSTRAINT uq_cache_symbol_type UNIQUE (symbol, asset_type)
);
```

Purpose: Provides a high-speed price cache table with automated expiration timestamps.

ER Diagram Check: ■ Matches

19. Create Audit Logs Table

```
CREATE TABLE audit_logs (
    log_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID NOT NULL,
    action VARCHAR(255) NOT NULL,
    timestamp TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT NULL,
    CONSTRAINT fk_audit_user FOREIGN KEY (user_id) REFERENCES users (user_id) ON
        DELETE CASCADE
);
```

Purpose: Maintains security and compliance audit trails of all user actions.

ER Diagram Check: ■ Matches

20. Create Relational Performance Indexes

```
CREATE INDEX idx_users_email ON users (email);
CREATE INDEX idx_assets_symbol ON assets (symbol);
CREATE INDEX idx_assets_type ON assets (type);
CREATE INDEX idx_portfolios_user_id ON portfolios (user_id);
CREATE INDEX idx_holdings_portfolio_id ON holdings (portfolio_id);
CREATE INDEX idx_holdings_symbol ON holdings (symbol);
CREATE INDEX idx_tx_portfolio_time ON transactions (portfolio_id, timestamp DESC);
CREATE INDEX idx_tx_user ON transactions (user_id);
CREATE INDEX idx_perf_portfolio_time ON portfolio_performance (portfolio_id,
    timestamp DESC);
CREATE INDEX idx_watchlist_items_watchlist ON watchlist_items (watchlist_id);
CREATE INDEX idx_alerts_user_enabled ON alerts (user_id, enabled);
CREATE INDEX idx_price_history_sym_time ON price_history (symbol, timestamp DESC);
CREATE INDEX idx_price_cache_expire ON price_cache (expire_at);
CREATE INDEX idx_audit_user_time ON audit_logs (user_id, timestamp DESC);
```

Purpose: Optimizes foreign key joins, lookups, and time-series query performance.

ER Diagram Check: ■ Matches

PART 2: INSERT, UPDATE, AND DELETE (DML) QUERIES

21. User Registration (Signup)

```
INSERT INTO users (email, password_hash, base_currency)
VALUES ('investor@example.com', '$2b$12$e8uq...', 'USD')
RETURNING user_id, email, base_currency, created_at;
```

Purpose: Inserts a newly registered user account and returns the generated user ID.

ER Diagram Check: ■ Matches

22. Update User Base Currency Preference

```
UPDATE users
SET base_currency = 'EUR'
WHERE user_id = '00000000-0000-0000-0000-000000000001';
```

Purpose: Updates the display and calculation currency code for a specific user.

ER Diagram Check: ■ Matches

23. Insert New Catalog Asset

```
INSERT INTO assets (symbol, name, type, network, contract_address,
    ticker_exchange, sector, exchange_id)
VALUES ('ETH', 'Ethereum', 'crypto', 'Ethereum Mainnet', NULL, NULL, NULL, NULL)
RETURNING asset_id, symbol;
```

Purpose: Inserts a new financial asset into the global assets catalog.

ER Diagram Check: ■ Matches

24. Upsert Ticker Statistics

```
INSERT INTO ticker_stats (asset_id, price_per_unit, volume_24h, high_24h,
    low_24h, updated_at)
VALUES ('22222222-2222-2222-2222-222222222222', 3250.50, 18500000000.00, 3310.00,
    3190.00, CURRENT_TIMESTAMP)
ON CONFLICT (asset_id)
DO UPDATE SET
    price_per_unit = EXCLUDED.price_per_unit,
    volume_24h = EXCLUDED.volume_24h,
    high_24h = EXCLUDED.high_24h,
    low_24h = EXCLUDED.low_24h,
    updated_at = CURRENT_TIMESTAMP;
```

Purpose: Inserts or updates 24-hour market summary statistics for an asset.

ER Diagram Check: ■ Matches

25. Create User Portfolio

```
INSERT INTO portfolios (user_id, name)
VALUES ('00000000-0000-0000-0000-000000000001', 'Tech Growth Portfolio')
RETURNING portfolio_id, user_id, name, created_at;
```

Purpose: Creates a new portfolio entity owned by the specified user.

ER Diagram Check: ■ Matches

26. Rename User Portfolio

```
UPDATE portfolios
SET name = 'Core Retirement Portfolio'
WHERE portfolio_id = '11111111-1111-1111-1111-111111111111'
  AND user_id = '00000000-0000-0000-0000-000000000001';
```

Purpose: Renames an existing portfolio while enforcing user-level authorization.

ER Diagram Check: ■ Matches

27. Delete User Portfolio

```
DELETE FROM portfolios
WHERE portfolio_id = '11111111-1111-1111-1111-111111111111'
  AND user_id = '00000000-0000-0000-0000-000000000001';
```

Purpose: Deletes a portfolio and cascades deletion to all embedded holdings and performance records.

ER Diagram Check: ■ Matches

28. Upsert Position with Weighted Average Cost Basis

```
INSERT INTO holdings (portfolio_id, asset_id, symbol, name, type, quantity,
    average_buy_price)
VALUES ('11111111-1111-1111-1111-111111111111',
    '22222222-2222-2222-2222-222222222222', 'BTC', 'Bitcoin', 'crypto', 0.75,
    62000.00)
ON CONFLICT (portfolio_id, symbol)
DO UPDATE SET
    quantity = holdings.quantity + EXCLUDED.quantity,
    average_buy_price = ((holdings.quantity * holdings.average_buy_price) + (
        EXCLUDED.quantity * EXCLUDED.average_buy_price)) / (
        holdings.quantity + EXCLUDED.quantity),
    updated_at = CURRENT_TIMESTAMP
RETURNING *;
```

Purpose: Inserts a new holding or aggregates quantity and recalculates the weighted average purchase price on conflict.

ER Diagram Check: ■ Matches

---

29. Update Holding Position

```
UPDATE holdings
SET quantity = 1.25, average_buy_price = 61500.00, updated_at = CURRENT_TIMESTAMP
WHERE holding_id = '33333333-3333-3333-3333-333333333333'
  AND portfolio_id = '11111111-1111-1111-1111-111111111111';
```

Purpose: Manually modifies the quantity and buy price of a specific holding within a portfolio.

ER Diagram Check: ■ Matches

---

30. Delete Holding Position

```
DELETE FROM holdings
WHERE holding_id = '33333333-3333-3333-3333-333333333333'
  AND portfolio_id = '11111111-1111-1111-1111-111111111111';
```

Purpose: Removes an asset position from a portfolio.

ER Diagram Check: ■ Matches

---

31. Record Trade Execution Log

```
INSERT INTO transactions (user_id, portfolio_id, asset_id, symbol, type,
    quantity, price_at_transaction, network_hash)
VALUES ('00000000-0000-0000-0000-000000000001',
    '11111111-1111-1111-1111-111111111111',
    '22222222-2222-2222-2222-222222222222', 'NVDA', 'BUY', 20.0, 125.50, NULL)
RETURNING transaction_id, timestamp;
```

Purpose: Inserts an immutable transaction record for an executed trade or asset transfer.

ER Diagram Check: ■ Matches

---

32. Record Periodic Portfolio Performance Snapshot

```
INSERT INTO portfolio_performance (portfolio_id, total_value, total_profit_loss,
    total_profit_loss_pct, timestamp)
VALUES ('11111111-1111-1111-1111-111111111111', 128500.00, 18500.00, 16.8182,
    CURRENT_TIMESTAMP);
```

Purpose: Saves a historical snapshot of portfolio valuation and aggregate profit/loss.

ER Diagram Check: ■ Matches

---

33. Create User Watchlist

```
INSERT INTO watchlists (user_id, name)
VALUES ('00000000-0000-0000-0000-000000000001', 'Crypto Blue Chips')
RETURNING watchlist_id, name;
```

Purpose: Creates a new custom watchlist container for a user.

ER Diagram Check: ■ Matches

---

34. Add Symbol to Watchlist

```
INSERT INTO watchlist_items (watchlist_id, symbol, asset_id)
VALUES ('55555555-5555-5555-5555-555555555555', 'SOL',
    '66666666-6666-6666-6666-666666666666')
ON CONFLICT (watchlist_id, symbol) DO NOTHING
RETURNING item_id;
```

Purpose: Adds an asset symbol to a specified user watchlist.

ER Diagram Check: ■ Matches

---

35. Remove Symbol from Watchlist

```
DELETE FROM watchlist_items
WHERE watchlist_id = '55555555-5555-5555-5555-555555555555' AND symbol = 'SOL';
```

Purpose: Deletes a tracked asset symbol from a user's watchlist.

ER Diagram Check: ■ Matches

---

36. Create Price Alert Rule

```
INSERT INTO alerts (user_id, symbol, asset_id, threshold, direction, enabled)
VALUES ('00000000-0000-0000-0000-000000000001', 'BTC',
    '22222222-2222-2222-2222-222222222222', 70000.00, 'ABOVE', TRUE)
RETURNING alert_id;
```

Purpose: Inserts a new price monitoring rule for a target asset and threshold.

ER Diagram Check: ■ Matches

---

37. Update Alert Status on Trigger

```
UPDATE alerts
SET triggered_at = CURRENT_TIMESTAMP, enabled = FALSE
WHERE alert_id = '44444444-4444-4444-4444-444444444444';
```

Purpose: Disables an alert and marks its triggered timestamp once criteria are met.

ER Diagram Check: ■ Matches

---

38. Delete Price Alert

```
DELETE FROM alerts
WHERE alert_id = '44444444-4444-4444-4444-444444444444'
  AND user_id = '00000000-0000-0000-0000-000000000001';
```

Purpose: Deletes a user price alert rule.

ER Diagram Check: ■ Matches

---

39. Insert Notification Log

```
INSERT INTO notifications (alert_id, user_id, channel, status)
VALUES ('44444444-4444-4444-4444-444444444444',
    '00000000-0000-0000-0000-000000000001', 'EMAIL', 'SENT')
RETURNING notification_id;
```

Purpose: Records an audit log entry for a dispatched user notification.

ER Diagram Check: ■ Matches

---

40. Insert Price History Point

```
INSERT INTO price_history (symbol, asset_id, price, volume, timestamp)
VALUES ('BTC', '22222222-2222-2222-2222-222222222222', 64320.00, 24500000000.00,
    CURRENT_TIMESTAMP);
```

Purpose: Inserts a historical price timestamp record for asset charting.

ER Diagram Check: ■ Matches

---

41. Insert News Article

```
INSERT INTO news_articles (article_id, headline, source, url, published_at)
VALUES ('77777777-7777-7777-7777-777777777777',
    'Fed Signals Steady Interest Rates', 'Reuters',
    'https://reuters.com/article1', CURRENT_TIMESTAMP);
```

Purpose: Inserts a new financial news publication record.

ER Diagram Check: ■ Matches

42. Link News Article to Asset

```
INSERT INTO news_article_assets (article_id, asset_id)
VALUES ('77777777-7777-7777-7777-777777777777',
    '22222222-2222-2222-2222-222222222222')
ON CONFLICT DO NOTHING;
```

Purpose: Maps a news article to an associated asset ticker.

ER Diagram Check: ■ Matches

43. Upsert Price Cache Entry

```
INSERT INTO price_cache (symbol, asset_type, price_usd, high_24h, low_24h,
    volume_24h, change_24h, fetched_at, expire_at, metadata)
VALUES ('BTC', 'crypto', 64200.00, 65000.00, 63800.00, 29000000000.00, 2.45,
    CURRENT_TIMESTAMP, CURRENT_TIMESTAMP + INTERVAL '5 minutes',
    '{"provider": "coingecko"}'::jsonb)
ON CONFLICT (symbol, asset_type)
DO UPDATE SET
    price_usd = EXCLUDED.price_usd,
    high_24h = EXCLUDED.high_24h,
    low_24h = EXCLUDED.low_24h,
    volume_24h = EXCLUDED.volume_24h,
    change_24h = EXCLUDED.change_24h,
    fetched_at = EXCLUDED.fetched_at,
    expire_at = EXCLUDED.expire_at,
    metadata = EXCLUDED.metadata;
```

Purpose: Updates or inserts external API quote results with a 5-minute expiration time.

ER Diagram Check: ■ Matches

44. Record User Audit Log

```
INSERT INTO audit_logs (user_id, action, timestamp)
VALUES ('00000000-0000-0000-0000-000000000001', 'USER_LOGIN', CURRENT_TIMESTAMP);
```

Purpose: Logs user security and configuration events for auditing.

ER Diagram Check: ■ Matches

PART 3: SELECT, ANALYTICAL, AND REPORTING QUERIES

45. User Authentication Credentials Lookup

```
SELECT user_id, email, password_hash, base_currency, created_at
FROM users
WHERE email = 'investor@example.com';
```

Purpose: Retrieves user credentials and security salt for password verification during login.

ER Diagram Check: ■ Matches

46. Fetch Current User Profile

```
SELECT u.user_id, u.email, u.base_currency, u.created_at, cr.rate_to_usd
FROM users u
LEFT JOIN currency_rates cr ON u.base_currency = cr.currency_code
WHERE u.user_id = '00000000-0000-0000-0000-000000000001';
```

Purpose: Fetches the authenticated user profile along with currency exchange rate details.

ER Diagram Check: ■ Matches

47. List User Portfolios with Total Position Counts

```
SELECT
    p.portfolio_id,
    p.name,
    p.created_at,
    COUNT(h.holding_id) AS total_positions
FROM portfolios p
LEFT JOIN holdings h ON p.portfolio_id = h.portfolio_id
WHERE p.user_id = '00000000-0000-0000-0000-000000000001'
GROUP BY p.portfolio_id, p.name, p.created_at
ORDER BY p.created_at DESC;
```

Purpose: Returns all portfolios owned by a user along with the count of active holdings.

ER Diagram Check: ■ Matches

48. Retrieve Single Portfolio by ID

```
SELECT portfolio_id, user_id, name, created_at
FROM portfolios
WHERE portfolio_id = '11111111-1111-1111-1111-111111111111'
  AND user_id = '00000000-0000-0000-0000-000000000001';
```

Purpose: Retrieves a specific portfolio while enforcing user ownership boundaries.

ER Diagram Check: ■ Matches

49. List All Holdings in a Specific Portfolio

```
SELECT
    holding_id,
    portfolio_id,
    symbol,
    name,
    type,
    quantity,
    average_buy_price,
    updated_at
FROM holdings
WHERE portfolio_id = '11111111-1111-1111-1111-111111111111'
ORDER BY symbol ASC;
```

Purpose: Fetches all holding positions belonging to a portfolio.

ER Diagram Check: ■ Matches

50. Get Specific Holding Position by ID

```
SELECT h.holding_id, h.portfolio_id, h.symbol, h.name, h.type, h.quantity,
    h.average_buy_price, h.updated_at
FROM holdings h
JOIN portfolios p ON h.portfolio_id = p.portfolio_id
WHERE h.holding_id = '33333333-3333-3333-3333-333333333333'
  AND p.user_id = '00000000-0000-0000-0000-000000000001';
```

Purpose: Retrieves single holding details ensuring the user owns the parent portfolio.

ER Diagram Check: ■ Matches

51. Real-Time Holdings Valuation with Unrealized Gain/Loss

```
SELECT
    h.holding_id,
    h.symbol,
    h.name,
    h.type,
    h.quantity,
    h.average_buy_price,
    (h.quantity * h.average_buy_price) AS total_invested,
    COALESCE(pc.price_usd, ts.price_per_unit, 0) AS current_price,
    (h.quantity * COALESCE(pc.price_usd, ts.price_per_unit, 0)) AS current_value,
    ((h.quantity * COALESCE(pc.price_usd, ts.price_per_unit,
        0)) - (h.quantity * h.average_buy_price)) AS gain_loss,
    CASE
        WHEN (h.quantity * h.average_buy_price) > 0
        THEN (((h.quantity * COALESCE(pc.price_usd, ts.price_per_unit,
            0)) - (h.quantity * h.average_buy_price)) / (
            h.quantity * h.average_buy_price)) * 100
        ELSE 0
    END AS gain_loss_pct
FROM holdings h
LEFT JOIN assets a ON h.symbol = a.symbol
LEFT JOIN ticker_stats ts ON a.asset_id = ts.asset_id
LEFT JOIN price_cache pc ON h.symbol = pc.symbol AND
    pc.expire_at > CURRENT_TIMESTAMP
WHERE h.portfolio_id = '11111111-1111-1111-1111-111111111111'
ORDER BY current_value DESC;
```

Purpose: Joins positions with cached and catalog prices to calculate current position value, unrealized profit/loss, and ROI %.

ER Diagram Check: ■ Matches

52. Portfolio Total Valuation & Base Currency Conversion

```
SELECT
    p.portfolio_id,
    p.name AS portfolio_name,
    u.base_currency,
    cr.rate_to_usd,
    SUM(h.quantity * h.average_buy_price) AS total_cost_usd,
    SUM(h.quantity * COALESCE(pc.price_usd, ts.price_per_unit,
        0)) AS total_market_value_usd,
    (SUM(h.quantity * COALESCE(pc.price_usd, ts.price_per_unit,
        0)) - SUM(h.quantity * h.average_buy_price)) AS total_unrealized_pnl_usd,
    CASE
        WHEN SUM(h.quantity * h.average_buy_price) > 0
        THEN ((SUM(h.quantity * COALESCE(pc.price_usd, ts.price_per_unit,
            0)) - SUM(h.quantity * h.average_buy_price)) / SUM(h.quantity *
            h.average_buy_price)) * 100
        ELSE 0
    END AS total_unrealized_pnl_pct,
    SUM(h.quantity * COALESCE(pc.price_usd, ts.price_per_unit,
        0)) * cr.rate_to_usd AS total_value_base_currency
FROM portfolios p
JOIN users u ON p.user_id = u.user_id
JOIN currency_rates cr ON u.base_currency = cr.currency_code
JOIN holdings h ON p.portfolio_id = h.portfolio_id
LEFT JOIN assets a ON h.symbol = a.symbol
LEFT JOIN ticker_stats ts ON a.asset_id = ts.asset_id
LEFT JOIN price_cache pc ON h.symbol = pc.symbol AND
    pc.expire_at > CURRENT_TIMESTAMP
WHERE p.portfolio_id = '11111111-1111-1111-1111-111111111111'
GROUP BY p.portfolio_id, p.name, u.base_currency, cr.rate_to_usd;
```

Purpose: Computes aggregate portfolio cost, market value, PnL, and converted value in user's base currency.

ER Diagram Check: ■ Matches

53. Asset Class Allocation Breakdown (Crypto vs Stock)

```
WITH PositionValues AS (
    SELECT
        h.type,
        SUM(h.quantity * COALESCE(pc.price_usd, ts.price_per_unit,
            0)) AS class_value
    FROM holdings h
    LEFT JOIN assets a ON h.symbol = a.symbol
    LEFT JOIN ticker_stats ts ON a.asset_id = ts.asset_id
    LEFT JOIN price_cache pc ON h.symbol = pc.symbol AND
        pc.expire_at > CURRENT_TIMESTAMP
    WHERE h.portfolio_id = '11111111-1111-1111-1111-111111111111'
    GROUP BY h.type
),
TotalValue AS (
    SELECT SUM(class_value) AS grand_total FROM PositionValues
)
SELECT
    pv.type AS asset_class,
    pv.class_value,
    ROUND((pv.class_value / NULLIF(tv.grand_total, 0)) * 100,
        2) AS allocation_percentage
FROM PositionValues pv, TotalValue tv;
```

Purpose: Calculates portfolio asset distribution percentage between crypto and equity asset classes.

ER Diagram Check: ■ Matches

54. Individual Asset Allocation Weighting

```
WITH PortfolioValuation AS (
    SELECT
        h.holding_id,
        h.symbol,
        h.type,
        (h.quantity * COALESCE(pc.price_usd, ts.price_per_unit,
            0)) AS market_value
    FROM holdings h
    LEFT JOIN assets a ON h.symbol = a.symbol
    LEFT JOIN ticker_stats ts ON a.asset_id = ts.asset_id
    LEFT JOIN price_cache pc ON h.symbol = pc.symbol AND
        pc.expire_at > CURRENT_TIMESTAMP
    WHERE h.portfolio_id = '11111111-1111-1111-1111-111111111111'
),
TotalValuation AS (
    SELECT SUM(market_value) AS total_market_value FROM PortfolioValuation
)
SELECT
    pv.symbol,
    pv.type,
    pv.market_value,
    ROUND((pv.market_value / NULLIF(tv.total_market_value, 0)) * 100,
        2) AS allocation_percentage
FROM PortfolioValuation pv, TotalValuation tv
ORDER BY pv.market_value DESC;
```

Purpose: Computes each asset's percentage contribution to the total portfolio value.

ER Diagram Check: ■ Matches

55. Fetch Paginated Portfolio Transaction History

```
SELECT
    t.transaction_id,
    t.symbol,
    t.type,
    t.quantity,
    t.price_at_transaction,
    (t.quantity * t.price_at_transaction) AS total_amount,
    t.network_hash,
    t.timestamp
FROM transactions t
WHERE t.portfolio_id = '11111111-1111-1111-1111-111111111111'
ORDER BY t.timestamp DESC
LIMIT 50 OFFSET 0;
```

Purpose: Returns paginated trade execution history for a portfolio sorted chronologically descending.

ER Diagram Check: ■ Matches

56. List User Watchlists with Live Market Prices

```
SELECT
    w.watchlist_id,
    w.name AS watchlist_name,
    wi.item_id,
    wi.symbol,
    wi.added_at,
    COALESCE(pc.price_usd, ts.price_per_unit, 0) AS current_price,
    pc.change_24h
FROM watchlists w
JOIN watchlist_items wi ON w.watchlist_id = wi.watchlist_id
LEFT JOIN assets a ON wi.symbol = a.symbol
LEFT JOIN ticker_stats ts ON a.asset_id = ts.asset_id
LEFT JOIN price_cache pc ON wi.symbol = pc.symbol
WHERE w.user_id = '00000000-0000-0000-0000-000000000001'
ORDER BY w.name ASC, wi.added_at DESC;
```

Purpose: Retrieves all user watchlists and items combined with 24-hour price and change data.

ER Diagram Check: ■ Matches

57. Active Price Alerts Trigger Evaluation

```
SELECT
    al.alert_id,
    al.user_id,
    al.symbol,
    al.threshold,
    al.direction,
    COALESCE(pc.price_usd, ts.price_per_unit) AS current_price,
    u.email
FROM alerts al
JOIN users u ON al.user_id = u.user_id
LEFT JOIN assets a ON al.symbol = a.symbol
LEFT JOIN ticker_stats ts ON a.asset_id = ts.asset_id
LEFT JOIN price_cache pc ON al.symbol = pc.symbol
WHERE al.enabled = TRUE
  AND (
      (al.direction = 'ABOVE' AND COALESCE(pc.price_usd,
          ts.price_per_unit) >= al.threshold)
      OR
      (al.direction = 'BELOW' AND COALESCE(pc.price_usd,
          ts.price_per_unit) <= al.threshold)
  );
```

Purpose: Evaluates current prices against enabled threshold conditions to determine alerts requiring dispatch.

ER Diagram Check: ■ Matches

58. Top Performing Holdings by Unrealized Return (%)

```
SELECT
    h.symbol,
    h.name,
    h.type,
    (h.quantity * COALESCE(pc.price_usd, ts.price_per_unit, 0)) AS current_value,
    (((COALESCE(pc.price_usd, ts.price_per_unit,
        0) - h.average_buy_price) / NULLIF(h.average_buy_price,
        0)) * 100) AS return_pct
FROM holdings h
JOIN portfolios p ON h.portfolio_id = p.portfolio_id
LEFT JOIN assets a ON h.symbol = a.symbol
LEFT JOIN ticker_stats ts ON a.asset_id = ts.asset_id
LEFT JOIN price_cache pc ON h.symbol = pc.symbol AND
    pc.expire_at > CURRENT_TIMESTAMP
WHERE p.user_id = '00000000-0000-0000-0000-000000000001'
ORDER BY return_pct DESC
LIMIT 5;
```

Purpose: Ranks top 5 highest returning holdings across all portfolios of a user.

ER Diagram Check: ■ Matches

59. 24-Hour Historical Performance Snapshot Series

```
SELECT
    pp.performance_id,
    pp.total_value,
    pp.total_profit_loss,
    pp.total_profit_loss_pct,
    pp.timestamp
FROM portfolio_performance pp
WHERE pp.portfolio_id = '11111111-1111-1111-1111-111111111111'
  AND pp.timestamp >= CURRENT_TIMESTAMP - INTERVAL '24 hours'
ORDER BY pp.timestamp ASC;
```

Purpose: Retrieves 24-hour timeseries data points for plotting portfolio valuation charts.

ER Diagram Check: ■ Matches

60. Asset News Feed for User Portfolio Holdings

```
SELECT DISTINCT
    na.article_id,
    na.headline,
    na.source,
    na.url,
    na.published_at,
    a.symbol
FROM news_articles na
JOIN news_article_assets naa ON na.article_id = naa.article_id
JOIN assets a ON naa.asset_id = a.asset_id
JOIN holdings h ON a.symbol = h.symbol
JOIN portfolios p ON h.portfolio_id = p.portfolio_id
WHERE p.user_id = '00000000-0000-0000-0000-000000000001'
ORDER BY na.published_at DESC
LIMIT 20;
```

Purpose: Fetches relevant market news articles filtered by assets actively held in user portfolios.

ER Diagram Check: ■ Matches

61. User Activity & Audit Log Trail

```
SELECT log_id, user_id, action, timestamp
FROM audit_logs
WHERE user_id = '00000000-0000-0000-0000-000000000001'
ORDER BY timestamp DESC
LIMIT 100;
```

Purpose: Retrieves recent security and transactional activity audit records for compliance.

ER Diagram Check: ■ Matches

PART 4: PL/PGSQL PROCEDURES, FUNCTIONS & TRIGGERS

62. Auto-Update Holdings Modification Timestamp Trigger

```
CREATE OR REPLACE FUNCTION update_holdings_timestamp()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    NEW.updated_at := CURRENT_TIMESTAMP;
    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_update_holdings_timestamp
BEFORE UPDATE ON holdings
FOR EACH ROW
EXECUTE FUNCTION update_holdings_timestamp();
```

Purpose: Automatically synchronizes the `updated_at` column with the current timestamp whenever a holding record is modified or updated.

ER Diagram Check: ■ Matches

63. Auto-Insert Transaction Activity into Audit Logs Trigger

```
CREATE OR REPLACE FUNCTION log_transaction_activity()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO audit_logs (
        log_id,
        user_id,
        action,
        timestamp
    ) VALUES (
        uuid_generate_v4(),
        NEW.user_id,
        'TRADE_EXECUTION: ' || NEW.type || ' ' || NEW.quantity::TEXT || ' ' ||
            NEW.symbol || ' @ ' || NEW.price_at_transaction::TEXT || ' (
            Portfolio: ' || NEW.portfolio_id::TEXT || ')',
        COALESCE(NEW.timestamp, CURRENT_TIMESTAMP)
    );
    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_log_transaction_activity
AFTER INSERT ON transactions
FOR EACH ROW
EXECUTE FUNCTION log_transaction_activity();
```

Purpose: Automatically writes an immutable audit record to `audit_logs` whenever a new trade or asset transfer is inserted into `transactions`.

ER Diagram Check: ■ Matches

64. Weighted-Average-Cost Holding Position Upsert

```
CREATE OR REPLACE FUNCTION upsert_holding(
    p_portfolio_id UUID,
    p_asset_id UUID,
    p_symbol VARCHAR,
    p_name VARCHAR,
    p_type VARCHAR,
    p_quantity NUMERIC,
    p_buy_price NUMERIC
)
RETURNS UUID
LANGUAGE plpgsql
AS $$
DECLARE
    v_holding_id UUID;
BEGIN
    -- Input validation
    IF p_quantity <= 0 THEN
        RAISE EXCEPTION 'Holding quantity must be strictly positive (
            received: %)', p_quantity;
    END IF;

    IF p_buy_price < 0 THEN
        RAISE EXCEPTION 'Holding purchase price cannot be negative (
            received: %)', p_buy_price;
    END IF;

    IF p_type NOT IN ('crypto', 'stock') THEN
        RAISE EXCEPTION 'Invalid asset type "%". Must be "crypto" or "stock"',
            p_type;
    END IF;

    -- Perform atomic upsert with weighted average cost recalculation
    INSERT INTO holdings (
        portfolio_id,
        asset_id,
        symbol,
        name,
        type,
        quantity,
        average_buy_price,
        updated_at
    ) VALUES (
        p_portfolio_id,
        p_asset_id,
        p_symbol,
        p_name,
        p_type,
        p_quantity,
        p_buy_price,
        CURRENT_TIMESTAMP
    )
    ON CONFLICT (portfolio_id, symbol)
    DO UPDATE SET
        quantity = holdings.quantity + EXCLUDED.quantity,
        average_buy_price = ((holdings.quantity * holdings.average_buy_price) + (
            EXCLUDED.quantity * EXCLUDED.average_buy_price)) / (
            holdings.quantity + EXCLUDED.quantity),
        updated_at = CURRENT_TIMESTAMP
    RETURNING holding_id INTO v_holding_id;

    RETURN v_holding_id;
EXCEPTION
    WHEN OTHERS THEN
        RAISE EXCEPTION 'Failed to upsert holding for symbol % in portfolio %:
            %', p_symbol, p_portfolio_id, SQLERRM;
END;
$$;
```

Purpose: Safely upserts an asset holding in a portfolio, recomputing its weighted-average cost basis and enforcing strict positive quantity and price validation rules within an exception block.

ER Diagram Check: ■ Matches

65. Atomic Transaction Recording and Holdings Synchronization

```
CREATE OR REPLACE FUNCTION record_transaction_and_update_holding(
    p_user_id UUID,
    p_portfolio_id UUID,
    p_asset_id UUID,
    p_symbol VARCHAR,
    p_type VARCHAR,
    p_quantity NUMERIC,
    p_price NUMERIC,
    p_network_hash VARCHAR DEFAULT NULL,
    p_asset_name VARCHAR DEFAULT NULL,
    p_asset_type VARCHAR DEFAULT NULL
)
RETURNS UUID
LANGUAGE plpgsql
AS $$
DECLARE
    v_tx_id UUID;
    v_current_qty NUMERIC(24, 8);
    v_current_avg NUMERIC(18, 6);
    v_resolved_name VARCHAR(150);
    v_resolved_type VARCHAR(20);
BEGIN
    -- Validate inputs
    IF p_quantity <= 0 THEN
        RAISE EXCEPTION 'Transaction quantity must be strictly greater than 0';
    END IF;

    IF p_price < 0 THEN
        RAISE EXCEPTION 'Transaction price cannot be negative';
    END IF;

    IF p_type NOT IN ('BUY', 'SELL', 'TRANSFER_IN', 'TRANSFER_OUT') THEN
        RAISE EXCEPTION 'Invalid transaction type: %. Expected BUY, SELL,
            TRANSFER_IN, or TRANSFER_OUT', p_type;
    END IF;

    -- Resolve asset metadata if not supplied
    IF p_asset_name IS NULL OR p_asset_type IS NULL THEN
        SELECT name, type INTO v_resolved_name, v_resolved_type
        FROM assets
        WHERE symbol = p_symbol
        LIMIT 1;

        v_resolved_name := COALESCE(v_resolved_name, p_symbol);
        v_resolved_type := COALESCE(v_resolved_type, 'crypto');
    ELSE
        v_resolved_name := p_asset_name;
        v_resolved_type := p_asset_type;
    END IF;

    -- Handle Holdings Adjustment
    IF p_type IN ('BUY', 'TRANSFER_IN') THEN
        INSERT INTO holdings (
            portfolio_id, asset_id, symbol, name, type, quantity,
                average_buy_price, updated_at
        ) VALUES (
            p_portfolio_id, p_asset_id, p_symbol, v_resolved_name,
                v_resolved_type, p_quantity, p_price, CURRENT_TIMESTAMP
        )
        ON CONFLICT (portfolio_id, symbol)
        DO UPDATE SET
            quantity = holdings.quantity + EXCLUDED.quantity,
            average_buy_price = CASE
                WHEN (holdings.quantity + EXCLUDED.quantity) > 0
                THEN ((holdings.quantity * holdings.average_buy_price) + (
                    EXCLUDED.quantity * EXCLUDED.average_buy_price)) / (
                    holdings.quantity + EXCLUDED.quantity)
                ELSE holdings.average_buy_price
            END,
            updated_at = CURRENT_TIMESTAMP;
```

```
    ELSIF p_type IN ('SELL', 'TRANSFER_OUT') THEN
        SELECT quantity, average_buy_price INTO v_current_qty, v_current_avg
        FROM holdings
        WHERE portfolio_id = p_portfolio_id AND symbol = p_symbol;

        IF NOT FOUND THEN
            RAISE EXCEPTION 'Cannot execute %: No existing position for symbol %
                in portfolio %', p_type, p_symbol, p_portfolio_id;
        END IF;

        IF v_current_qty < p_quantity THEN
            RAISE EXCEPTION 'Insufficient position balance: attempted to % % %,
                but current holding is %', p_type, p_quantity, p_symbol,
                v_current_qty;
        ELSIF v_current_qty = p_quantity THEN
            DELETE FROM holdings
            WHERE portfolio_id = p_portfolio_id AND symbol = p_symbol;
        ELSE
            UPDATE holdings
            SET quantity = quantity - p_quantity,
                updated_at = CURRENT_TIMESTAMP
            WHERE portfolio_id = p_portfolio_id AND symbol = p_symbol;
        END IF;
    END IF;

    -- Insert Transaction Ledger Record
    INSERT INTO transactions (
        user_id,
        portfolio_id,
        asset_id,
        symbol,
        type,
        quantity,
        price_at_transaction,
        network_hash,
        timestamp
    ) VALUES (
        p_user_id,
        p_portfolio_id,
        p_asset_id,
        p_symbol,
        p_type,
        p_quantity,
        p_price,
        p_network_hash,
        CURRENT_TIMESTAMP
    )
    RETURNING transaction_id INTO v_tx_id;

    RETURN v_tx_id;
EXCEPTION
    WHEN OTHERS THEN
        RAISE EXCEPTION 'Transaction execution failed and state rolled back: %',
            SQLERRM;
END;
$$;
```

Purpose: Atomically executes a trade order by validating position balances, updating or deleting rows in `holdings` with weighted-average cost adjustments, and inserting the ledger entry into `transactions` in a single transaction-safe operation.

ER Diagram Check: ■ Matches

66. Active Price Alerts Evaluation and Notification Dispatch

```
CREATE OR REPLACE FUNCTION check_price_alerts()
RETURNS INTEGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_alert RECORD;
    v_triggered_count INTEGER := 0;
    v_current_price NUMERIC(18, 6);
BEGIN
    FOR v_alert IN
        SELECT
            al.alert_id,
            al.user_id,
            al.symbol,
            al.threshold,
            al.direction,
            COALESCE(pc.price_usd, ts.price_per_unit) AS current_price
        FROM alerts al
        LEFT JOIN assets a ON al.symbol = a.symbol
        LEFT JOIN ticker_stats ts ON a.asset_id = ts.asset_id
        LEFT JOIN price_cache pc ON al.symbol = pc.symbol
        WHERE al.enabled = TRUE
    LOOP
        BEGIN
            v_current_price := v_alert.current_price;

            IF v_current_price IS NOT NULL THEN
                IF (v_alert.direction = 'ABOVE' AND
                    v_current_price >= v_alert.threshold) OR
                   (v_alert.direction = 'BELOW' AND
                       v_current_price <= v_alert.threshold) THEN

                    -- Insert Notification Record
                    INSERT INTO notifications (
                        alert_id,
                        user_id,
                        channel,
                        status,
                        sent_at
                    ) VALUES (
                        v_alert.alert_id,
                        v_alert.user_id,
                        'IN_APP',
                        'PENDING',
                        CURRENT_TIMESTAMP
                    );

                    -- Deactivate Alert & Record Trigger Timestamp
                    UPDATE alerts
                    SET enabled = FALSE,
                        triggered_at = CURRENT_TIMESTAMP
                    WHERE alert_id = v_alert.alert_id;

                    v_triggered_count := v_triggered_count + 1;
                END IF;
            END IF;
        EXCEPTION
            WHEN OTHERS THEN
                -- Isolate failure to single alert so entire evaluation loop
                    completes
                RAISE WARNING 'Error evaluating alert ID % for symbol %: %',
                    v_alert.alert_id, v_alert.symbol, SQLERRM;
        END;
    END LOOP;

    RETURN v_triggered_count;
END;
$$;
```

Purpose: Iterates through all active price alerts, compares real-time asset market prices against user thresholds using procedural control flow, dispatches queued notification records, and deactivates triggered alerts while isolating exceptions.

ER Diagram Check: ■ Matches

67. Live Portfolio Valuation and Unrealized Profit/Loss Table

```
CREATE OR REPLACE FUNCTION get_portfolio_valuation(p_portfolio_id UUID)
RETURNS TABLE (
    holding_id UUID,
    symbol VARCHAR(20),
    name VARCHAR(150),
    type VARCHAR(20),
    quantity NUMERIC(24, 8),
    average_buy_price NUMERIC(18, 6),
    current_price NUMERIC(18, 6),
    cost_basis NUMERIC(24, 6),
    market_value NUMERIC(24, 6),
    unrealized_pnl NUMERIC(24, 6),
    unrealized_pnl_pct NUMERIC(18, 4),
    portfolio_base_currency VARCHAR(10),
    currency_rate_to_usd NUMERIC(18, 6),
    market_value_base_currency NUMERIC(24, 6)
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT
        h.holding_id,
        h.symbol,
        h.name,
        h.type,
        h.quantity,
        h.average_buy_price,
        COALESCE(pc.price_usd, ts.price_per_unit, 0.0) AS current_price,
        (h.quantity * h.average_buy_price) AS cost_basis,
        (h.quantity * COALESCE(pc.price_usd, ts.price_per_unit,
            0.0)) AS market_value,
        ((h.quantity * COALESCE(pc.price_usd, ts.price_per_unit,
            0.0)) - (h.quantity * h.average_buy_price)) AS unrealized_pnl,
        CASE
            WHEN h.average_buy_price > 0
            THEN (((COALESCE(pc.price_usd, ts.price_per_unit,
                0.0) - h.average_buy_price) / h.average_buy_price) * 100.0)
            ELSE 0.0
        END AS unrealized_pnl_pct,
        u.base_currency AS portfolio_base_currency,
        COALESCE(cr.rate_to_usd, 1.0) AS currency_rate_to_usd,
        ((h.quantity * COALESCE(pc.price_usd, ts.price_per_unit,
            0.0)) / COALESCE(cr.rate_to_usd, 1.0)) AS market_value_base_currency
    FROM holdings h
    JOIN portfolios p ON h.portfolio_id = p.portfolio_id
    JOIN users u ON p.user_id = u.user_id
    LEFT JOIN currency_rates cr ON u.base_currency = cr.currency_code
    LEFT JOIN assets a ON h.symbol = a.symbol
    LEFT JOIN ticker_stats ts ON a.asset_id = ts.asset_id
    LEFT JOIN price_cache pc ON h.symbol = pc.symbol AND
        pc.expire_at > CURRENT_TIMESTAMP
    WHERE h.portfolio_id = p_portfolio_id
    ORDER BY market_value DESC;
END;
$$;
```

Purpose: Computes comprehensive real-time position valuations, cost basis, unrealized gain/loss amounts and percentages, and foreign currency conversion for all holdings in a portfolio.

ER Diagram Check: ■ Matches

68. Automated Portfolio Valuation Performance Snapshot Recorder

```
CREATE OR REPLACE FUNCTION record_portfolio_snapshot(p_portfolio_id UUID)
RETURNS UUID
LANGUAGE plpgsql
AS $$
DECLARE
    v_total_value NUMERIC(18, 2) := 0.0;
    v_total_cost NUMERIC(18, 2) := 0.0;
    v_total_pnl NUMERIC(18, 2) := 0.0;
    v_total_pnl_pct NUMERIC(8, 4) := 0.0;
    v_perf_id UUID;
BEGIN
    -- Calculate current portfolio valuation aggregates
    SELECT
        COALESCE(SUM(h.quantity * COALESCE(pc.price_usd, ts.price_per_unit,
            0.0)), 0.0)::NUMERIC(18, 2),
        COALESCE(SUM(h.quantity * h.average_buy_price), 0.0)::NUMERIC(18, 2)
    INTO v_total_value, v_total_cost
    FROM holdings h
    LEFT JOIN assets a ON h.symbol = a.symbol
    LEFT JOIN ticker_stats ts ON a.asset_id = ts.asset_id
    LEFT JOIN price_cache pc ON h.symbol = pc.symbol AND
        pc.expire_at > CURRENT_TIMESTAMP
    WHERE h.portfolio_id = p_portfolio_id;

    v_total_pnl := v_total_value - v_total_cost;

    IF v_total_cost > 0 THEN
        v_total_pnl_pct := ((v_total_pnl / v_total_cost) * 100.0)::NUMERIC(8, 4);
    ELSE
        v_total_pnl_pct := 0.0;
    END IF;

    -- Record snapshot in portfolio_performance
    INSERT INTO portfolio_performance (
        portfolio_id,
        total_value,
        total_profit_loss,
        total_profit_loss_pct,
        timestamp
    ) VALUES (
        p_portfolio_id,
        v_total_value,
        v_total_pnl,
        v_total_pnl_pct,
        CURRENT_TIMESTAMP
    )
    RETURNING performance_id INTO v_perf_id;

    RETURN v_perf_id;
EXCEPTION
    WHEN OTHERS THEN
        RAISE EXCEPTION 'Failed to record performance snapshot for portfolio %:
            %', p_portfolio_id, SQLERRM;
END;
$$;
```

Purpose: Aggregates all current asset holding values in a portfolio and persists a point-in-time equity and performance record to `portfolio_performance` for historical tracking.

ER Diagram Check: ■ Matches

69. Expire and Purge Stale Price Cache Routine

```
CREATE OR REPLACE FUNCTION expire_price_cache()
RETURNS INTEGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_deleted_count INTEGER := 0;
BEGIN
    DELETE FROM price_cache
    WHERE expire_at < CURRENT_TIMESTAMP;

    GET DIAGNOSTICS v_deleted_count = ROW_COUNT;

    RETURN v_deleted_count;
EXCEPTION
    WHEN OTHERS THEN
        RAISE EXCEPTION 'Price cache expiration routine failed: %', SQLERRM;
END;
$$;
```

Purpose: Housekeeping cleanup routine that purges stale market quote rows from `price_cache` whose TTL timestamp has elapsed, returning the total count of pruned records.

ER Diagram Check: ■ Matches

70. Recalculate 24-Hour Ticker Statistics from Price History

```
CREATE OR REPLACE FUNCTION refresh_ticker_stats(p_asset_id UUID)
RETURNS BOOLEAN
LANGUAGE plpgsql
AS $$
DECLARE
    v_symbol VARCHAR(20);
    v_latest_price NUMERIC(18, 6);
    v_high_24h NUMERIC(18, 6);
    v_low_24h NUMERIC(18, 6);
    v_volume_24h NUMERIC(24, 2);
BEGIN
    -- Find symbol for given asset
    SELECT symbol INTO v_symbol
    FROM assets
    WHERE asset_id = p_asset_id;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Asset with ID % does not exist', p_asset_id;
    END IF;

    -- Extract aggregates from price_history over the last 24 hours
    SELECT
        MAX(ph.price),
        MIN(ph.price),
        COALESCE(SUM(ph.volume), 0.0)
    INTO v_high_24h, v_low_24h, v_volume_24h
    FROM price_history ph
    WHERE (ph.asset_id = p_asset_id OR ph.symbol = v_symbol)
      AND ph.timestamp >= CURRENT_TIMESTAMP - INTERVAL '24 hours';

    -- Fetch the most recent recorded price
    SELECT price INTO v_latest_price
    FROM price_history ph
    WHERE (ph.asset_id = p_asset_id OR ph.symbol = v_symbol)
    ORDER BY ph.timestamp DESC
    LIMIT 1;

    -- If no recent history exists, maintain current or fallback
    IF v_latest_price IS NOT NULL THEN
        INSERT INTO ticker_stats (
            asset_id,
            price_per_unit,
            volume_24h,
            high_24h,
            low_24h,
            updated_at
        ) VALUES (
            p_asset_id,
            v_latest_price,
            COALESCE(v_volume_24h, 0.0),
            COALESCE(v_high_24h, v_latest_price),
            COALESCE(v_low_24h, v_latest_price),
            CURRENT_TIMESTAMP
        )
        ON CONFLICT (asset_id)
        DO UPDATE SET
            price_per_unit = EXCLUDED.price_per_unit,
            volume_24h = EXCLUDED.volume_24h,
            high_24h = EXCLUDED.high_24h,
            low_24h = EXCLUDED.low_24h,
            updated_at = CURRENT_TIMESTAMP;

        RETURN TRUE;
    END IF;

    RETURN FALSE;
EXCEPTION
    WHEN OTHERS THEN
        RAISE EXCEPTION 'Failed to refresh ticker stats for asset %: %',
            p_asset_id, SQLERRM;
```

```
END;
$$;
```

Purpose: Computes rolling 24-hour high, low, volume, and last price statistics from historical timeseries data points in `price_history` and upserts the calculated summary into `ticker_stats`.

ER Diagram Check: ■ Matches